



Extrakce dat z webu

Aka web scraping

doc. Ing. Radek Burget, Ph.D.

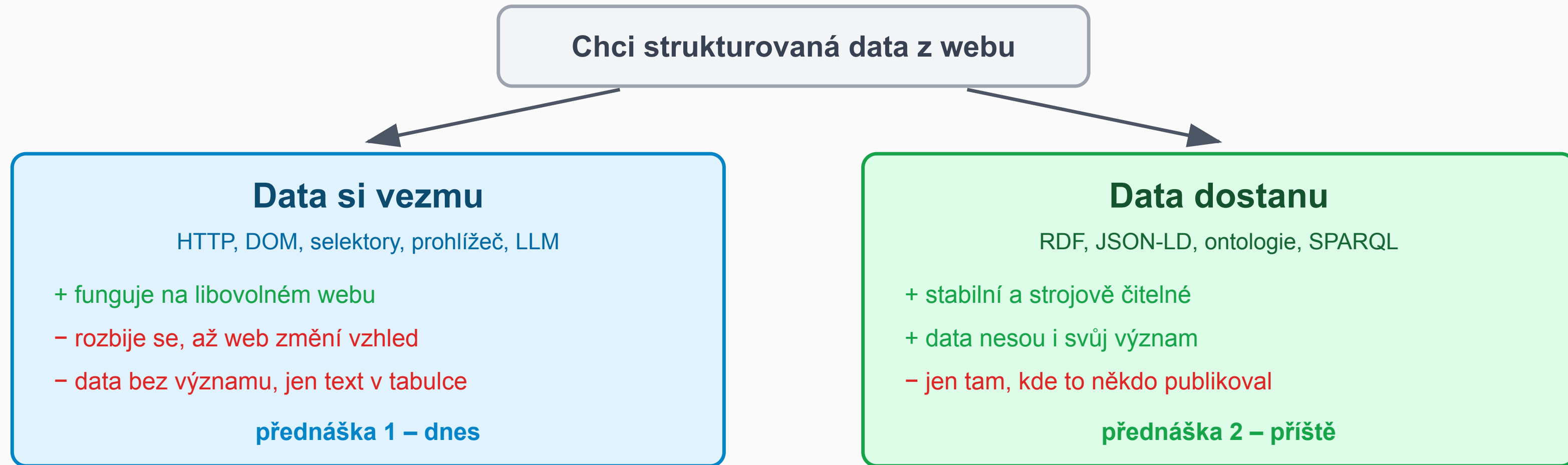
burgetr@fit.vutbr.cz

Kde jsme



- Web jako významný zdroj dat (různými způsoby)

Dvě cesty ke stejným datům



Dva protipóly téhož problému: násilím to jde vždycky, dohodou to vydrží.

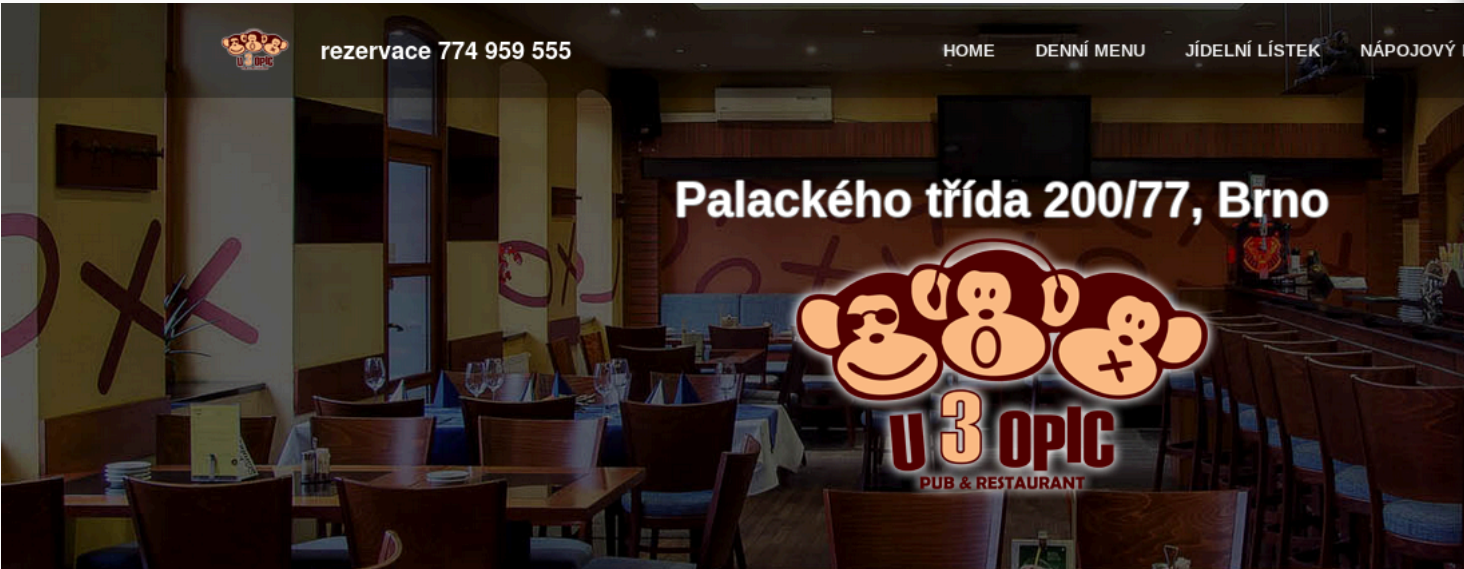
Motivace

- **Na webu je spousta dat** ukrytých ve webových stránkách
- Potřebujeme dále zpracovat tato data v počítačových aplikacích
 - Propojení s vlastními daty
 - Agregace – spojení výsledků z různých zdrojů
 - Analýza – statistiky, získávání znalostí
 - ... a mnohé další
- Potřebujeme **strukturovaná data**
 - Reprezentovatelná tabulkami relační databáze
 - Nebo alespoň XML, JSON, apod. *s pevnou strukturou*

Data na webu

- Webové stránky nejsou silně strukturované
- Nejčastěji v HTML
 - Prvotním cílem je *vizuální prezentace*
 - Kód je druhotný, *podřízený prvnímu cíli*
 - Není určen k dalšímu zpracování
 - Často proměnlivé schéma
- **Slabě strukturované dokumenty**

Vzhled a kód



```
<section class="menu">
  <div class="container">
    <div class="row">
      <div class="col-md-12">
        <div class="page-header wow fadeInDown">
          <h1>Dnešní polední menu</h1>
        </div>
      </div>
    </div>
    <div class="food-menu wow fadeInUp">
      <div class="row menu-items">
        <div class="menu-item col-sm-12 col-xs-12 starter"><div class="clearfix menu-wrapp
        </div>
      </div>
      <div class="row">
        <div class="col-md-12">
          <div class="menu-btn">
            <a class="btn btn-default btn-lg" href="/denni-menu/" role="button">Zobraz
          </div>
        </div>
      </div>
    </div>
  </div>
</section>
```

DNEŠNÍ POLEDNÍ MENU

Hrachová polévka s opečenou houskou

1. Indické butter chicken se smaženou cizrnou, rýže Basmati 149 Kč

2. Smažený česnekový řízek z vepřové kotlety, bramborový salát s majonézou 159 Kč

3. Barbecue Pulled Pork Hot Dog, smažené bramborové hranolky (máslová houska plněná trhaným vepřovým masem se sýrem
čedar a křupavou cibulkou) 179 Kč

ZOBRAZIT MENU

Další zdroje

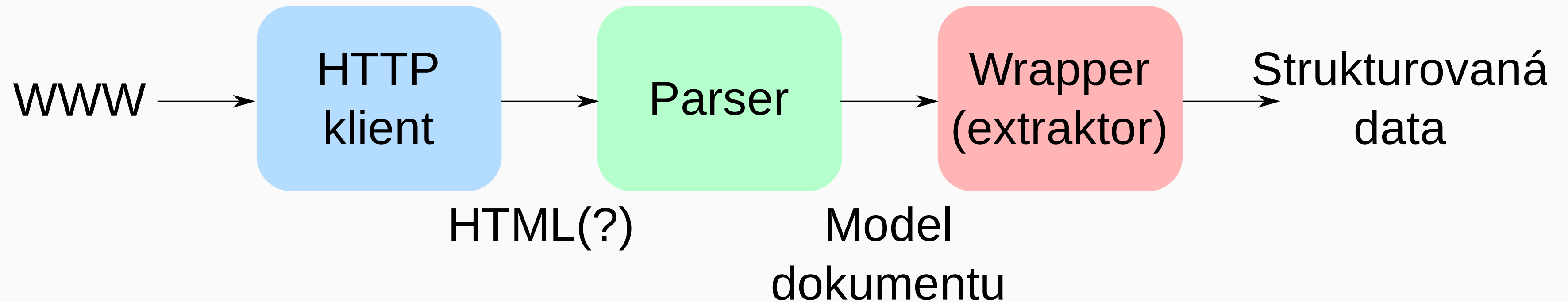
- Komerční
 - e-shopy, realitní servery, letenky, sportovní výsledky, sledování konkurence
- Výsledky vyhledávání
 - Např. hlídání pozice
- Veřejné rejstříky a instituce
 - jízdní řády, obchodní rejstřík, statistický úřad, weby zastupitelstev
- Kontrola reklamy
- ∞ dalších

Dílčí problémy

- Získání zdrojových dat
 - Jak stáhnout potřebné dokumenty z WWW (aby obsahovaly to, co mají?)
 - Paralelizace
- **Nalezení a extrakce dat**
 - Identifikace požadovaných údajů ve stránce
- Uložení výsledků
 - Může jich být **opravdu mnoho**
 - A mají nepravidelnou strukturu – viz přednášky o ukládání dat

Architektura

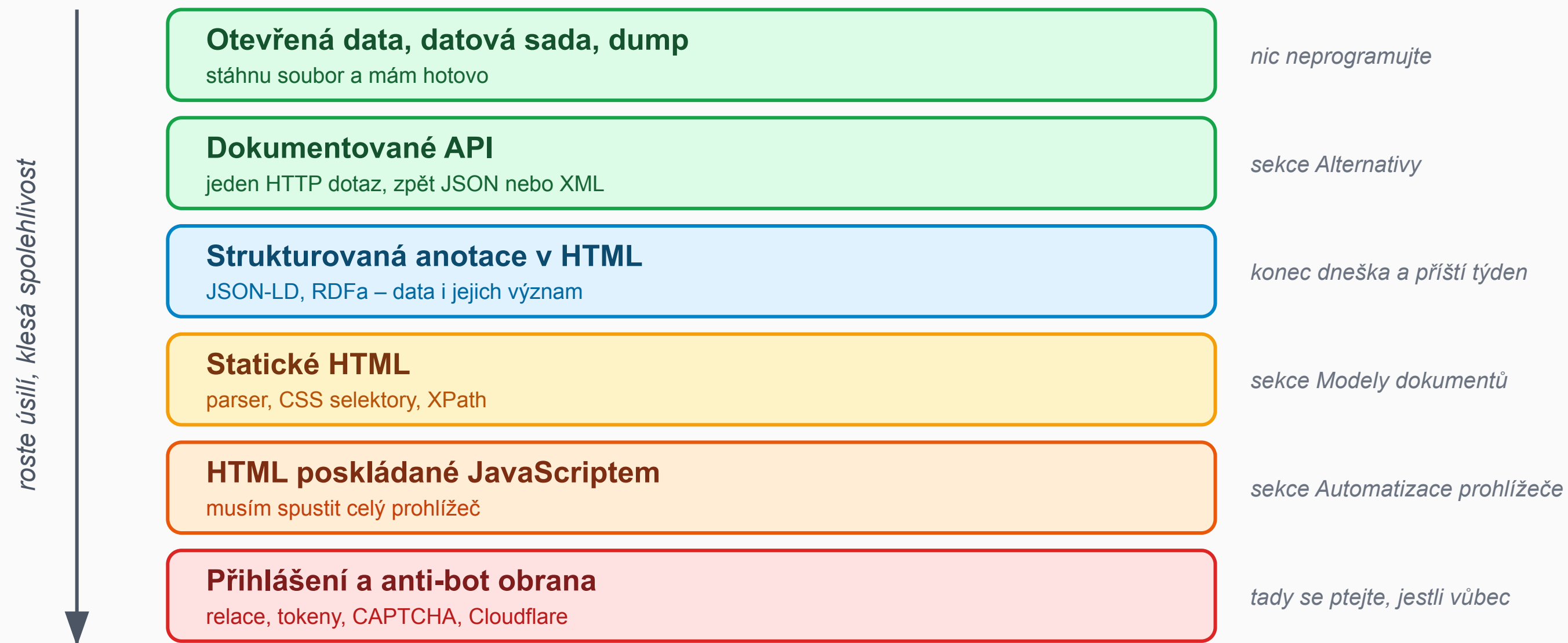
Základní architektura



Kde vzít data

Scraping je jedna z možností. Vyplatí se zkusit i ostatní.

Než začnete scrapovat



Otevřená data a API – Česko

- **ARES** – ekonomické subjekty, REST API vracející JSON ([dokumentace](#))
- **ČNB** – denní kurzy devizového trhu jako prostý textový soubor
- **NKOD** – [data.gov.cz](#), katalog českých otevřených dat
 - Má i [SPARQL endpoint](#) – příští týden pochopíte, co to je
- **ČSÚ** ([vdb.czso.cz](#)), [volby.cz](#) (XML), [Registr smluv](#) (XML, XSD)
- [Hlídač státu](#), [Golemio](#) (PID a data Prahy), [ČHMÚ](#), [data.europa.eu](#)

Jeden řádek místo scraperu

```
ARES=https://ares.gov.cz/ekonomicke-subjekty-v-be/rest/ekonomicke-subjekty  
curl -s $ARES/00216305 | jq '.obchodniJmeno, .sidlo.textovaAdresa'
```

```
"Vysoké učení technické v Brně"  
"Antonínská 548/1, Veverčí, 60200 Brno"
```

- Nebo ještě jednodušeji – **ani to nemusí být HTML nebo JSON**

```
curl -s https://www.cnb.cz/cs/financni-trhy/devizovy-trh/\kurzy-devizoveho-trhu/kurzy-devizoveho-trhu/denni_kurz.txt
```

```
14.09.2026 #177  
země|měna|množství|kód|kurz
```

Pokrytí v UPA

Zdroje a související problematika v UPA

Typ dat	Zdroj	Doběhne v přednášce
Grafová, propojená	Wikidata (SPARQL), DBpedia	2 sémantický web, 9 grafové DB
Prostorová	OpenStreetMap / Overpass API, RÚIAN, Golemio	12 prostorové DB
Časové řady	ČNB kurzy, ČHMÚ, polohy vozů PID	7 a 9 NoSQL, sloupcové
Objemná, textová	Common Crawl, OpenAlex, arXiv	13 vícedimenzionální indexy
Hierarchická	volby.cz (XML), Registr smluv	11 XML a JSON v databázích
Nepravidelná, špinavá	e-shopy, jídelníčky, sportovní výsledky	4 a 5 čištění, transformace

- Poslední řádek – projekty 1 a 2

A když nic z toho nestačí

- **Zdroj žádné API nemá**
 - Typicky menší weby, obecní úřady, jídelníčky, rozvrhy
- **API existuje, ale nevrací to, co potřebujete**
 - Jiná granularita, chybějící atributy, limit na počet záznamů
- **Potřebujete spojit víc zdrojů dohromady**
 - A aspoň jeden z nich je obyčejná webová stránka

Můžeme to zjistit přímo z HTML?

Quick & Dirty

Když potřebujeme rychle a jednorázově data z jednoduššího webu a nikdo se nás nebude ptát, jak jsme to udělali.

Shell je přítel

Motivace: [Evolution of a programmer](#)

Co se hodí, než začneme programovat:

- `wget, curl`
- `cat, grep, sed, cut`
- `awk` (jen pro fajnšmekry :-)
- `jq`, když zdroj vrací JSON – viz předchozí sekce

```
wget https://www.fit.vut.cz/study/courses/ -O out.html  
cat out.html | grep 'list-links__link' | sed 's/<[^<>]*>/;/g' | sed 's/;;*/;/g'  
cat data.csv | cut -f2 -d';'
```

Totéž v pythonu

```
import urllib.request
import re

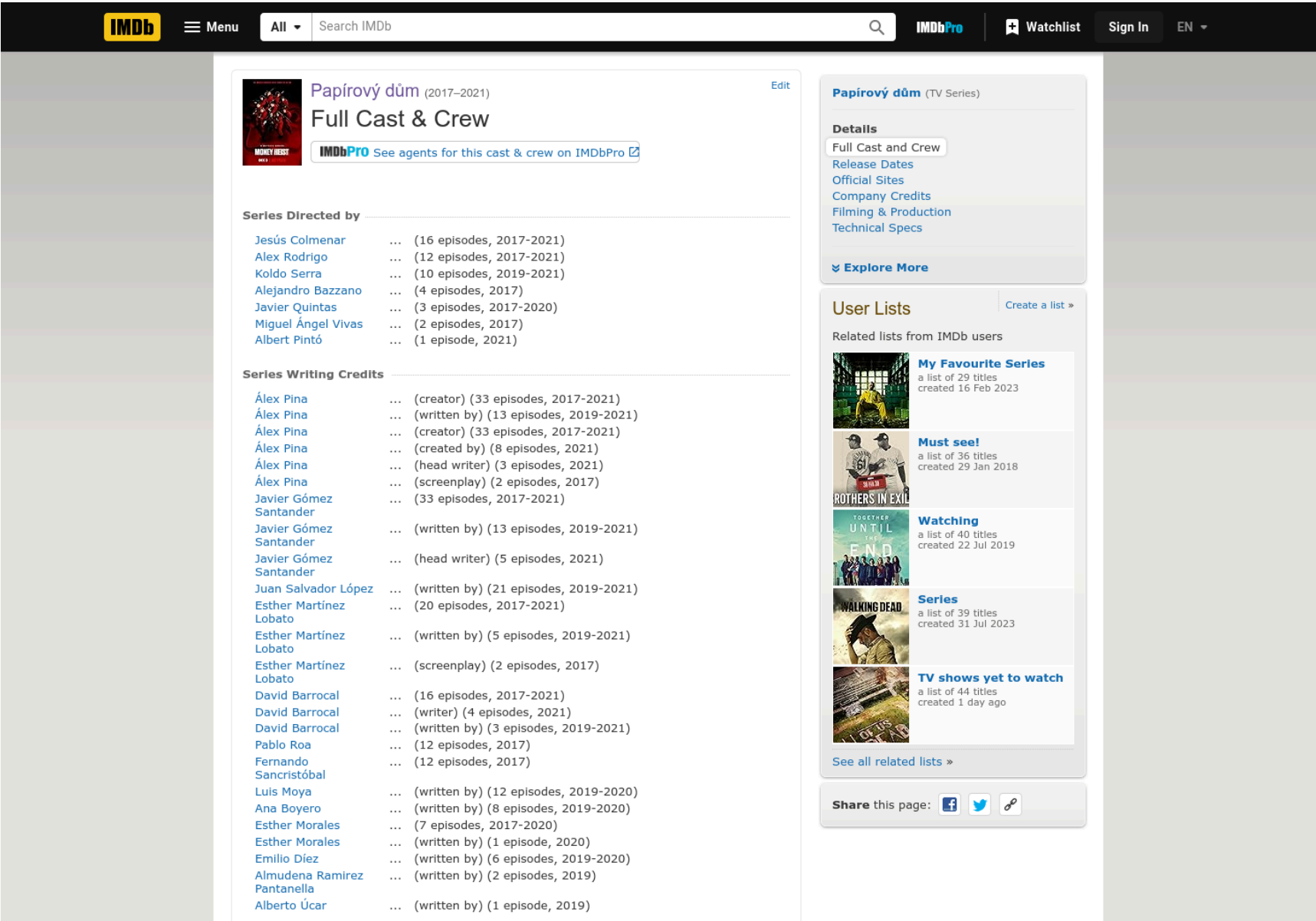
fid = urllib.request.urlopen('https://www.fit.vut.cz/study/courses/')
webpage = fid.read().decode('utf-8')
for line in webpage.split('\n'):
    if ('list-links__link') in line:
        line = re.sub(r"<[^<>]*>", ";", line);
        line = re.sub(r";;*;", ";", line);
        print(line)
```

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class Courses {

    public static void main(String[] args) throws Exception {
        var client = HttpClient.newHttpClient();
        var req = HttpRequest.newBuilder(
            URI.create("https://www.fit.vut.cz/study/courses/")).build()
```

Omezení: složitá struktura stránky



Zdrojová stránka – regex?!

Omezení: data nejsou v HTML



Inside UCICycling for AllParaWomen'sDevelopmentCycling Esports

CENTRE MONDIAL DU CYCLISME

WORLD CYCLING CENTRE

ROADTRACKMOUNTAIN BIKEBMX RACINGBMX FREESTYLETRIALSCYCLO-CROSSINDOOR

MOUNTAIN BIKE

OverviewAboutNewsCalendarEvents▼TeamsResultsRankingsRules and regulations

Rankings

RACE TYPECATEGORYSEASON

AllMen Elite2020



Cross-country Ranking Men Elite

Individual Ranking

Last Update
06/10/2020

Leader

 **SCHURTER Nino (SSR)**
1989 points

2nd rank

 **AVANCINI Henrique (CFR)**
1945 points

3rd rank

 **KERSCHBAUMER Gerhard (BTU)**
1339 points

Nation Ranking

Last Update
06/10/2020

Leader

 **SWITZERLAND**
4411 points

2nd rank

 **FRANCE**
3675 points

3rd rank

 **ITALY**
3153 points

UPA -- Extrakce dat z webu

21 / 69

Omezení: stejné URL, jiný obsah

Rankings

Cross-country Ranking Men Elite

SEASON2020

DATEMost Recent Ranking

EXPORT RANKINGS

Individual Ranking

Ranking Date06/10/2020

Leader

 SCHURTER Nino (SSR)

1989 points

Nation Ranking

Ranking Date06/10/2020

Leader

 SWITZERLAND

4411 points

NAME

TEAM

NATION

All

Rank	Rider	Nation	Team	Age	Points
1	SCHURTER Nino	 SUI	SCOTT - SRAM MTB RACING	34	1989
2	AVANCINI Henrique	 BRA	CANNONDALE FACTORY RACING	31	1945
3	KERSCHBAUMER Gerhard	 ITA	TORPADO URSUS	29	1339
4  +1	KORETZKY Victor	 FRA	KMC - ORBEA	26	1318
5  +3	VADER Milan	 NED	KMC - ORBEA	24	1279
6  -2	FLUECKIGER Mathias	 SUI	THÖMUS RN SWISS BIKE TEAM	32	1172
7  -1	TEMPIER Stephane	 FRA	TREK FACTORY RACING XC	34	1141
8  -1	SARROU Jordan	 FRA	ABSOLUTE - ABSALON - BMC	28	1096
9  +3	CINK Ondřej	 CZE	KROSS RACING TEAM	30	988
10  +4	COOPER Anton	 NZL	TREK FACTORY RACING XC	26	971
11  -1	FORSTER Lars	 SUI	SCOTT - SRAM MTB RACING	27	950

Omezení: přihlášení a přesměrování

CAS FIT VUT Login

Zadejte [FIT login](#) a [heslo](#) a Łukněte na Log In.
Please enter your FIT login and password and click the "Log In".

Přihlášení si vyžádala aplikace/Login requested by application **cosign-FIT-Email**

FIT Login

FIT Password

Need [an account or password help?](#)

Pokud jste tak ještě neučinili, [nainstalujte si certifikát CA VUT](#)

[CAS FIT](#) je "Centrální Autentizační Server" FIT pro Webové aplikace. Zajiřřuje spoleřnou autentizaci pro Webové aplikace na řůzných serverech. Pro ověřenř autentizace používá "Cookie" (ukládání Cookie musí být povoleno v prohlřžeči alespoň pro doměnu .fit.vutbr.cz).

Vaře IP adresa: 147.229.12.203

- Přihlašovací formulář s přesměrováním (a možným zabezpečením proti strojovému vyplnění)
- Zvládnutelné, ale komplikované

Modely dokumentů

Když regulární výrazy nestačí.

Modely HTML dokumentů

- Řetězec znaků
 - Jednoduchá implementace, rychlost, škálovatelnost
 - Regulární výrazy, HLRT wrappery
- Řetězec tokenů
 - Lexikální analyzátor rozpozná tagy, entity, text, apod.
 - HLRT wrappery
- Hierarchické modely
 - Nejčastěji DOM
 - Případně „odlehčená“ varianta

Wrapper

- Uvažujeme n datových polí, které se mají extrahovat z dokumentu
- Různé třídy wrapperů: LR, HLRT, ...
- HLRT wrapper:
 - **Head** – podřetězec před datovým blokem
 - **Left** – levý oddělovač (pro každé datové pole)
 - **Right** – pravý oddělovač (pro každé datové pole)
 - **Tail** – podřetězec za datovým blokem

$$wrapper = (h, t, l_1, r_1, l_2, r_2, \dots, l_n, r_n)$$

Řetězce tokenů

- Událostmi řízené parsery, např. `html.parser` v pythonu

```
from html.parser import HTMLParser

class MyHTMLParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print("Encountered a start tag:", tag)

    def handle_endtag(self, tag):
        print("Encountered an end tag :", tag)

    def handle_data(self, data):
        print("Encountered some data  :", data)
```

<https://docs.python.org/3/library/html.parser.html>

Document Object Model

- HTML kód reprezentovaný jako **strom objektů**
- Různé typy objektů (viz [přednášky IIS](#))

HTML Element (prvek)

- Úsek dokumentu vymezený *značkami* (tagy)

```
<p>Obsah elementu</p>
```

```
<div class="menu" id="mainmenu">  
Obsah elementu<br> Další obsah elementu.  
</div>
```

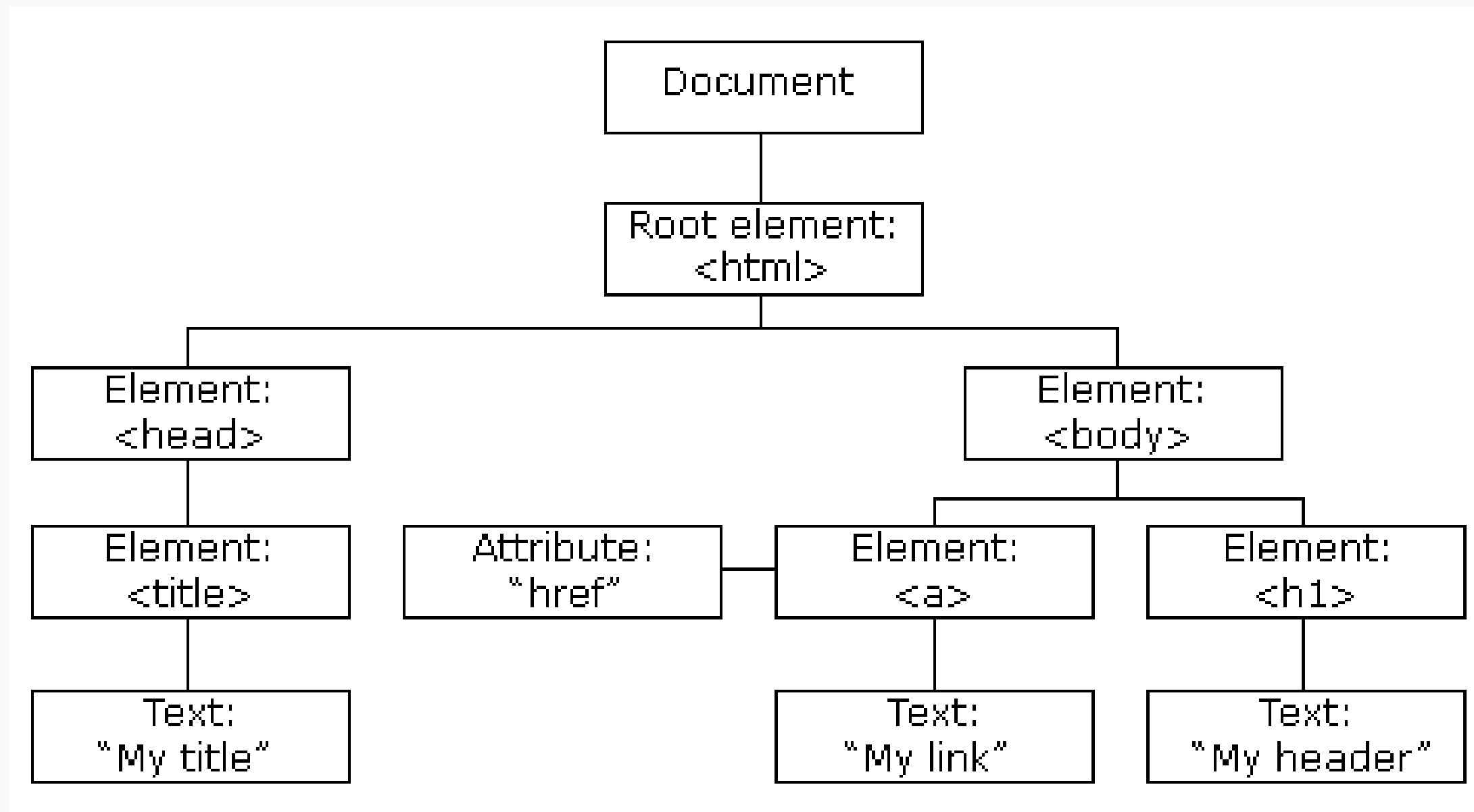
```
<div>Nějaký <em>zvýrazněný</em> text.</div>
```

- Vždy právě jeden *kořenový element*

DOM strom

- Kořenový uzel je typu **Document**
- Má jednoho potomka typu **Element** ~ Document element (root)
- Element může mít potomky typu
 - **Element** – vnořené prvky
 - **text** – textový obsah, vždy listový uzel
 - výjimečně jiné (např. **Entity**)

DOM strom



https://www.w3schools.com/whatis/whatis_htmlidom.asp

Navigace v DOM stromu

- Standardní metody DOM tříd `Document` a `Element`
 - Vyhledání elementů: `querySelector()`, `querySelectorAll()`
 - Starší varianty: `getElementById()`, `getElementsByTagName()`
 - Navigace ve stromu: `parentNode`, `childNodes`, ...
 - Přístup k obsahu: `textContent`
- CSS selektory
 - Výsledkem je vždy *množina elementů*
 - `#main header .info`
- XPath
 - Výsledkem je také *množina elementů*

XPath

- Původně pro XML dokumenty, ale podporováno i některými knihovnamy pro HTML
- Oproti CSS složitější syntaxe, ale více možností:
 - Obecný výraz pro vlastnosti elementu v `[]` zahrnující hodnoty atributů, pořadí elementu a další
 - Navigace různými směry („osy“)
- Viz např. [Dokumentace na MDN](#)

```
var res = document.evaluate('//head/title', document.documentElement,  
    null, XPathResult.ANY_TYPE, null);  
console.log(res.iterateNext().textContent);
```

Praktické použití DOM

- Parserů podle specifikace HTML 5 je dnes dost
 - Python: [lxml.html](#), [html5lib](#); JavaScript: [parse5](#), [jsdom](#), [linkedom](#)
- Volba je **kompromis mezi přesností a rychlostí**
 - [html5lib](#) chybné HTML opraví přesně jako prohlížeč, ale je pomalý
 - [lxml](#), [selectolax](#) jsou řádově rychlejší za cenu odchylek u rozbitého kódu
- V praxi nejčastěji knihovny s pohodlným vlastním rozhraním
 - Python: [BeautifulSoup](#)
 - Java: [jSoup](#)
 - JavaScript: [cheerio](#)

BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

html = requests.get("https://www.fit.vut.cz/study/courses/").text
soup = BeautifulSoup(html, "lxml")
rows = soup.select("#list")[0].find_all("tr")
for row in rows:
    cells = row.find_all('td')
    out = "";
    for cell in cells:
        out = out + cell.text + ";"
    print(out)
```

```
Document doc = Jsoup.connect("https://en.wikipedia.org/").get();
log(doc.title());
Elements newsHeadlines = doc.select("#mp-itn b a");
for (Element headline : newsHeadlines) {
    log("%s\n\t%s",
        headline.attr("title"), headline.absUrl("href"));
}
```

- Také disponuje podmnožinou standardního DOM rozhraní

cheerio

```
import * as cheerio from 'cheerio';

const res = await fetch('https://www.fit.vut.cz/study/courses/');
const $ = cheerio.load(await res.text());

$('#list tr').each((i, tr) => {
  let line = "";
  $(tr).children().each((j, td) => {
    line += $(td).text() + ';';
  });
  console.log(line);
});
```

Automatizace prohlížeče

Když se ke stránce nedá dostat na jeden HTTP GET.

Levnější mezistupeň

- Spousta „dynamických“ stránek celý prohlížeč nepotřebuje
- **HTTP relace** – `requests.Session()` nebo `httplib` v pythonu
 - Drží cookies, umí přesměrování i přihlášení formulářem
- [Mechanical Soup](#) – nadstavba nad [BeautifulSoup](#)
 - „Klikání na odkazy“ – zjištění cíle a generování GET požadavku
 - „Vyplnění formulářů“ – zjištění `action` a `method`, odeslání hodnot
- JavaScript ale nespustí ani jedno

Playwright

- Prohlížeč dálkově ovládaný z kódu
<https://playwright.dev>
- Podpora více prohlížečů
 - Chromium/Chrome, Firefox, WebKit
- Více vývojových platforem
 - Node.js, Java, Python, .NET
- **Locator API s automatickým čekáním**
 - `page.locator("#list tr")` počká, až prvek existuje a je použitelný
 - Odpadá většina časových souběhů, kvůli kterým dřív scrapery náhodně selhávaly
- Vykonání JS kódu v prohlížeči (`page.evaluate()`)

Playwright – příklad

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.goto("https://www.fit.vut.cz/study/courses/")

    for row in page.locator("#list tbody tr").all():
        cells = row.locator("td").all_inner_texts()
        print(";".join(c.strip() for c in cells))

    browser.close()
```

- playwright codegen <adresa> – rovnou generuje kód
 - Nejrychlejší cesta od „tohle potřebuju“ k funkčnímu scraperu

Puppeteer

- Starší sourozenec, Chrome ovládaný z node.js
<https://pptr.dev>
- Velmi podobné API a možnosti
 - Ale jen Chrome/Chromium a jen JavaScript
 - Playwright založil původní vývojový tým Puppeteeru

Automatizace prohlížeče: Pros & Cons

- + Možnost navigace
- + Pohodlná extrakce dat
 - DOM, CSS selektory, XPath, jakýkoliv JavaScript
- + Stránka se vidí tak, jak ji vidí uživatel
- - Časově i prostorově náročné řešení
 - Spouští se celý prohlížeč – řádově pomalejší než jeden HTTP dotaz
- - Náročné ošetření vnějších podmínek
 - Regionální verze stránek, cookie lišty, A/B testy
- - Obtížnější ladění
 - Část kódu běží v node.js či pythonu, část v prohlížeči (různá prostředí)

Crawling ve velkém

Vratíme se k prvnímu dílčímu problému: **jak vůbec ty dokumenty získat?**

- Fronta URL (*frontier*) a evidence už navštívených adres
- **Deduplikace** – tatáž stránka pod různými URL ([?utm_source=](#), řazení, stránkování)
- Zdvořilostní prodleva mezi požadavky na tentýž server
- Cache a podmíněné požadavky, ať nestahujete totéž znovu
- Paralelizace **napříč doménami** (ne uvnitř jedné)
- [sitemap.xml](#) – web sám může poskytnout seznam stránek
- Hotové rámce: [Scrapy](#) (Python), [Crawlee](#) (Node.js, Python)
 - Řeší frontu, opakování, rate limiting i export dat

Smím to vůbec?

Technicky vás nic nezastaví. To ale není totéž jako svolení.

robots.txt

- Dohoda, ne zámeček – **je to prosba, ne technická překážka**
- Web tu říká, co si přeje, aby roboti nestahovali

```
# www.fit.vut.cz/robots.txt
User-agent: *
Disallow: /search/
Request-rate: 5/1s

User-agent: GPTBot
Disallow: /
```

```
# www.sreality.cz/robots.txt
User-agent: *
Disallow: /

#      neplecha ukončena

User-agent: Googlebot
Allow: /
```

- **Rozlišuje, kdo jste, ne co děláte** – Googlebot smí, vy ne

Co smím a co ne

Use of any device, tool, or process designed to data mine or scrape the content using automated means is prohibited without prior written permission from IMDb.

– *imdb.com/robots.txt*, a tahle přednáška IMDb používá jako příklad

- **Podmínky užití** jsou smlouva; [robots.txt](#) je konvence, ne zákon
- **Autorské právo** k obsahu – texty, fotografie, popisy produktů
- **Osobní údaje** – GDPR platí i na údaje veřejně dostupné
- Směrnice EU 2019/790
- **Veřejně dostupné ≠ volně použitelné**

Slušný scraper

- **Představte se** – vlastní `User-Agent` včetně kontaktu na sebe
- **Chodte pomalu** – limit požadavků, exponenciální backoff
 - Respektujte `429 Too Many Requests` a hlavičku `Retry-After`
- **Nestahujte totéž dvakrát** – cache, podmíněné požadavky (`If-Modified-Since`)
- **Berte jen to, co potřebujete** – ne celý web kvůli jedné tabulce
- **Paralelizujte napříč doménami**, ne uvnitř jedné

Anti-bot obrana (Cloudflare, CAPTCHA, fingerprinting) je jasný vzkaz: **tady vás nechceme**. Její obcházení je technicky i právně jiná liga.

Alternativy

Je analýza HTML kódu jediná možnost?

Využití API

- Některé stránky načítají zajímavý obsah dynamicky JavaScriptem
 - `XMLHttpRequest` nebo `fetch()`
 - (aka AJAX)
- Zdrojem dat je *HTTP endpoint*, který typicky vrátí
 - Útržky HTML kódu
 - *Serializovaná strukturovaná data* – JSON, XML, ...

Opět např. [žebříček UCI](#) – podívejte se do záložky **Network** v developer tools

Využití API: Pros & Cons

- Mírně jednodušší přístup k datům
 - Obvykle stačí jeden HTTP dotaz (GET nebo POST)
 - Parsujeme strukturovaný formát
- Formát dat může být ještě proměnlivější, než webová stránka
 - Čistě interní formát tvůrců aplikace
- Snaha komplikovat přístup třetích stran
 - Autorizační tokeny apod.
- Existují veřejné endpointy s dobře dokumentovaným formátem dat
 - Např. [Portál otevřených dat EU](#) nebo [NKOD](#)

Anotace webových stránek

Data pro stroje **přímo ve stránce**, vedle dat pro člověka.

- **JSON-LD** – dnes nejrozšířenější, samostatný blok v `<head>`

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": "Quip Kids Electric Toothbrush",
  "offers": { "@type": "Offer", "price": "25.00", "priceCurrency": "USD" }
}
</script>
```

- **Microdata** a [RDFa](#) – anotace rozptýlená v attributech existujících elementů
- [Microformats](#) – historicky první pokus, anotace hodnotami `class`

Proč je to pro nás dobrá zpráva

- Údaje jsou **explicitně pojmenované** – žádné hádání podle pozice v tabulce
- Identifikace objektů a vlastností pomocí **URI**
 - Celá řada slovníků (*ontologií*) pro různé domény, např. schema.org
- Umožňuje transformaci HTML na *linked data* reprezentovaná pomocí RDF
- Kdo to publikuje? Každý, kdo chce vypadat dobře ve výsledcích vyhledávání

Co ty URI znamenají, odkud se berou slovníky a jak se v takových datech dotazuje – [příští přednáška](#).

Když za nás pracuje model

Současná realita ve zpracování webu

Nedávný stav

- Manuální programování, aka **Webscrapping**
 - Ruční tvorba programů, které z HTML kódu extrahují, co je třeba
- Platformy pro zpřístupnění obsahu dokumentů přes API
 - Integrace ručně vytvořených scraperů
 - **Přístup „manufaktura“**



Inteligentní extrakce

Tzn. bez „ruční práce“ v podobě hledání elementů, regulárních výrazů, CSS selektorů, XPath výrazů, apod.

1. Jazykové modely

- Text dokumentu nebo kód jako součást promptu

2. Strojové učení

- „Naučení“ extraktoru na anotovaných příkladech

3. Modelem řízená extrakce

- Specifikace předpokládané struktury dat (ER diagram?, *ontologie*, ...)
- Nalezení výskytu požadovaných skupin dat ve zdrojové stránce

Jazykové modely

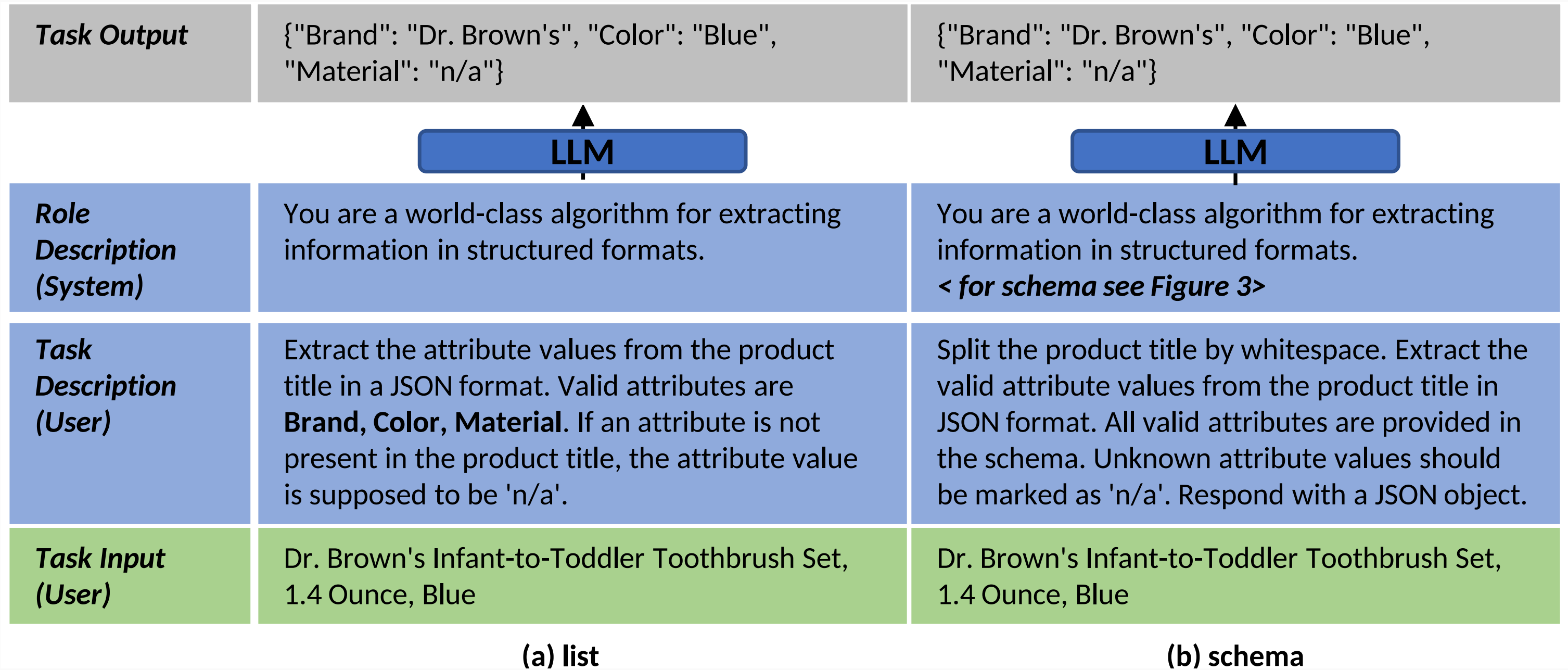
- Předáme text nebo kód dokumentu a ptáme se

Je-li toto stránka produktu, jak se tento produkt jmenuje a kolik stojí?

- **Zero-shot** – pouze instrukce, žádné příklady
- **One (few)-shot** – příklady zdrojových dat a očekávaných výsledků
- Specifikace cílového formátu odpovědi
 - Instrukce (použij JSON, názvy atributů) nebo rovnou schéma

A. Brinkmann et al.: [ExtractGPT: Exploring the Potential of Large Language Models for Product Attribute Value Extraction](#)

Příklad promptu



Strukturovaný výstup

- Formát odpovědi dnes nemusíme *prosit*, můžeme ho **vynutit schématem**
 - Model fyzicky nemůže vygenerovat token, který by schéma porušil

```
{  
  "type": "object",  
  "properties": {  
    "nazev": { "type": "string" },  
    "cena": { "type": "number" },  
    "mena": { "type": "string", "enum": ["CZK", "EUR"] },  
    "skladem": { "type": "boolean" }  
  },  
  "required": ["nazev", "cena"]  
}
```

- Odpadá parsování odpovědi a ošetřování „skoro JSONu“

Cena a jak ji srazit

- Zdrojové HTML má běžně stovky kB a z velké části jsou to skripty a styly
 - **Předzpracování** – zahodit `<script>`, `<style>`, navigaci, převést na text
 - Poslat jen **relevantní podstrom**, nalezený lacino selektorem
 - Cache a dávkové zpracování
- Hlavní problém je **nedeterminismus a ověřitelnost**
 - Dva běhy nad stejnou stránkou, dva různé výsledky
 - Jak poznáte, že model tiše vynechal tři řádky tabulky?

Hybridní vzor

- Zeptáme se modelu **jednou**: *vygeneruj selektory pro tuhle stránku*
- Pak běží obyčejný **deterministický scraper**
 - Levný, rychlý, opakovatelný, snadno testovatelný
- Model zavoláme znovu, **teprve až scraper přestane fungovat**
 - Změna webu je událost, ne každodenní stav

AI agenti a MCP

- LLM nejen analyzuje vstup, ale i **řídí nástroje**
 - Popíšeme schopnosti nástrojů, model generuje sekvenci volání
- **MCP** (Model Context Protocol) – standard pro připojení nástrojů k modelu
 - Nástroj napsaný jednou funguje s libovolným klientem
 - Viz přednáška PIS [Alternativní technologie a architektury](#)
 - Existuje **Playwright MCP** – prohlížeč jako nástroj modelu
- Agenti typu *browser-use / computer-use* – model stránku vidí a klikáním se proklikává
- Knihovny pro slepení, např. [LangChain](#)
- Nedeterminismus z předchozího slajdu tu platí dvojnásob

A co dál

Co zatím ani jazykové modely neumí spolehlivě.

Strojové učení – scénář

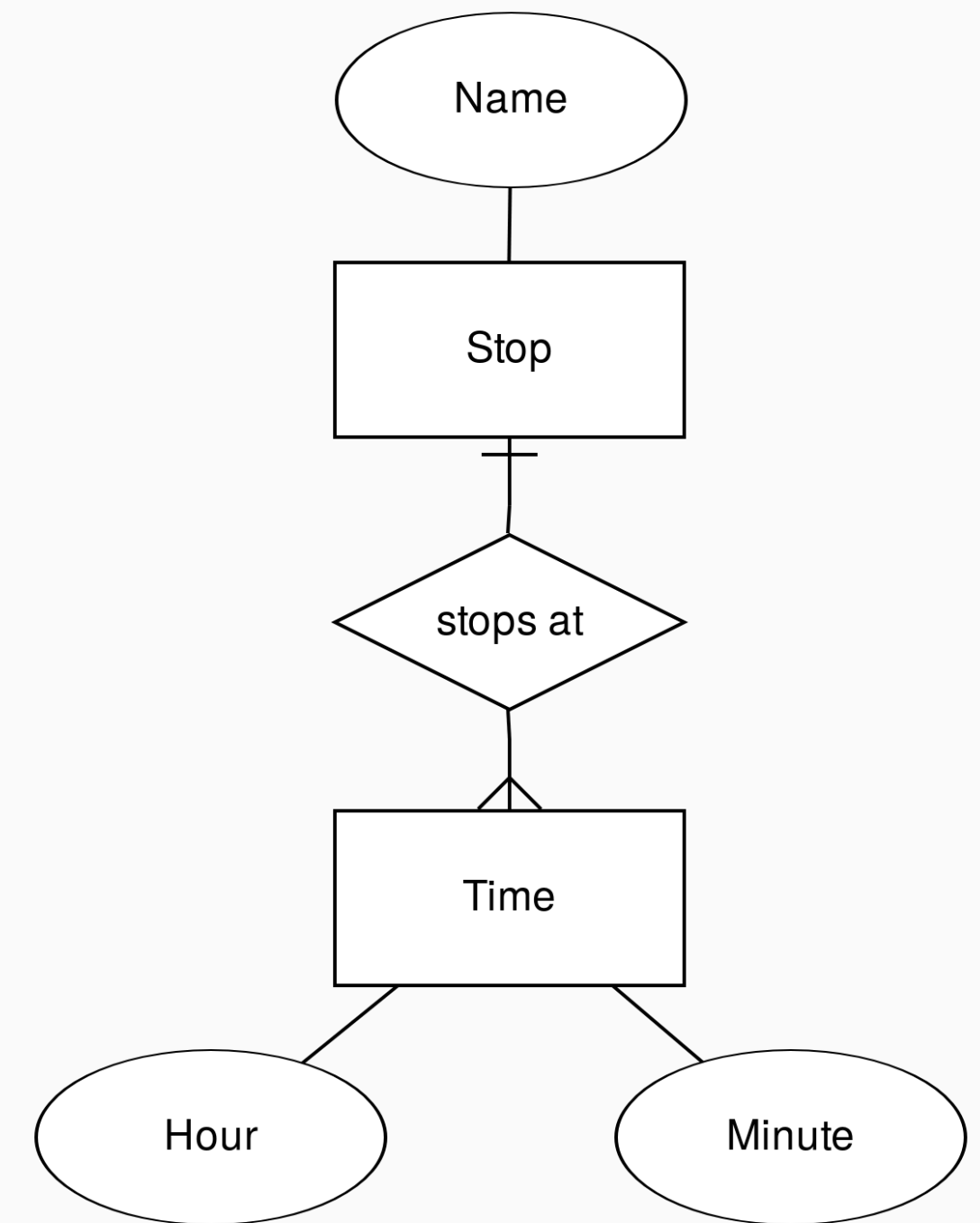
- Trénovací množina dokumentů
 - Obvykle dokumenty z jednoho zdroje
 - Anotace částí obsahu, které se mají extrahovat
 - Odvození pravidel pro extrakci
- Množina nových, neznámých dokumentů
 - Extrakce dat na základě odvozených pravidel

Strojové učení – metody

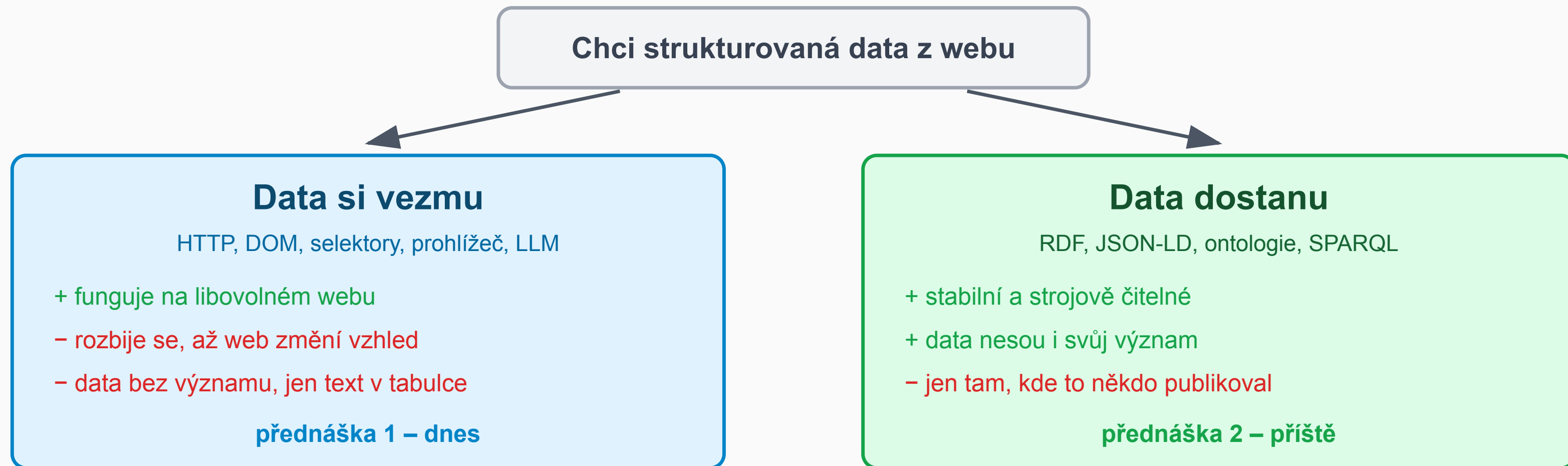
- Sekvenční modely stránky (znaky, tokeny)
 - Inference gramatik (*wrapper induction*), skryté Markovovy modely, ...
- Hierarchické modely
 - Zobecněný DOM (odstranění implementačních detailů)
 - Stromové automaty
- Vizuální modely dokumentů
 - Segmentace stránek
 - Klasifikace na základě vizuálních rysů

Modelem řízená extrakce

- Vstup: Entity, atributy, vztahy
- Přibližné rozpoznání jednotlivých údajů
 - Regulární výrazy
 - Klasifikace textu nebo vizuálních vlastností
 - Mapování na databázi
- Nalezení datových záznamů
 - Využití pravidelnosti, opakující se vzory
- Cíl: **garantovat**, že výsledek sedí na schéma a že se nic neztratilo



Příště: data dohodu



Dva protipóly téhož problému: násilím to jde vždycky, dohodou to vydrží.

Dnes jsme si data **brali**. Příště: Jak je publikovat **dobrovolně a strojově čitelně**.

Tři řádky místo tří set

Největší česká města i s počtem obyvatel – **bez scrapování**:

```
SELECT ?mestoLabel ?obyvateL WHERE {  
  ?mesto wdt:P31 wd:Q5153359 ;          # je statutární město  
        wdt:P1082 ?obyvateL .          # má počet obyvatel  
  SERVICE wikibase:label { bd:serviceParam wikibase:language "cs". }  
}  
ORDER BY DESC(?obyvateL)
```

Praha	1397880
Brno	402739
Ostrava	283187
Plzeň	187928

- Zkuste na query.wikidata.org

A to je vše!

Demo kód:

<https://github.com/DIFS-Teaching/webscraping>

Dotazy?